Name: Kirsten Kajubi
Course: CIS 310
Assignment 3: 8 bit processor
Date: 04/20/2026

Github Repo Link: https://github.com/KirstenKajubi/Assignment-3-8-bit-processor

Design and Implementation of an 8-Bit Multicycle Processor

1. Introduction

In this project, I designed and implemented an 8-bit multicycle processor using Digital logic simulation. The goal was to build a functional CPU capable of executing simple arithmetic instructions while demonstrating proper coordination between datapath components and a control unit.

The processor follows a multicycle architecture, where each instruction is executed over four stages: fetch, decode, execute, and writeback. This design emphasizes the importance of timing, synchronization, and control signal generation in processor design.

2. Instruction Format

The processor uses an 8-bit instruction format divided into two fields:

- Upper 4 bits: Opcode
- Lower 4 bits: Immediate value

Example:

0x12 = 0001 0010
↑ ↑
opcode immediate

In this implementation, the opcode `0001` represents the ADD instruction.

3. Processor Architecture

The processor consists of the following components:

3.1 Program Counter (PC)

- Stores the address of the current instruction
- Increments only during the fetch stage
- Controlled using the signal `PC_inc`

3.2 Instruction Memory (ROM)

- Stores the program instructions
- Uses a 3-bit address space (8 instructions total)
- Outputs 8-bit instruction values

3.3 Instruction Register (IR)

- Stores the current instruction
- Loads only during the fetch stage (`IR_load`)
- Keeps the instruction stable throughout execution

3.4 Control Unit

- Implements a 4-stage multicycle control:
  - Fetch
  - Decode
  - Execute
  - Writeback
- Built using a one-hot state machine with flip-flops
- Uses a decoder to interpret opcode
- Generates control signals:
  - `IR_load`
  - `PC_inc`
  - `add_execute`
  - `add_writeback`

3.5 Arithmetic Logic Unit (ALU)

- Performs arithmetic operations (addition in this design)
- Inputs:
  - A: Current register value
  - B: Zero-extended immediate value
- Output:
  - Result sent to the register

3.6 Register

- Stores the result of computations
- Updates only during the writeback stage
- Controlled by:
    - `en` (enable signal from `add_writeback`)
    - `clk`

4. Datapath Overview

The datapath is structured as follows:

PC → ROM → IR → Control Unit
IR → ALU → Register → ALU (feedback loop)

Operation flow:

- The PC selects an instruction from ROM
- The instruction is loaded into the IR
- The control unit decodes the instruction
- The ALU performs computation
- The result is stored in the register and fed back for the next operation

5. Control Logic Design

The control unit operates using a one-hot state machine:

Cycle 1: inst_fetch
Cycle 2: inst_decode
Cycle 3: inst_exec
Cycle 4: write_back

Control signals are defined as:

- IR_load = inst_fetch
- PC_inc = inst_fetch
- add_execute = inst_exec AND (opcode == ADD)
- add_writeback = write_back AND (opcode == ADD)

This ensures proper sequencing and synchronization between control and datapath components.

6. Program Execution

The program loaded into ROM is:

ADD 2
ADD 3
ADD 1
ADD 4

Machine code representation:

0x12
0x13
0x11
0x14

7. Results

The register output (`RegQ`) was monitored during execution of the program.

The processor produced the following sequence:

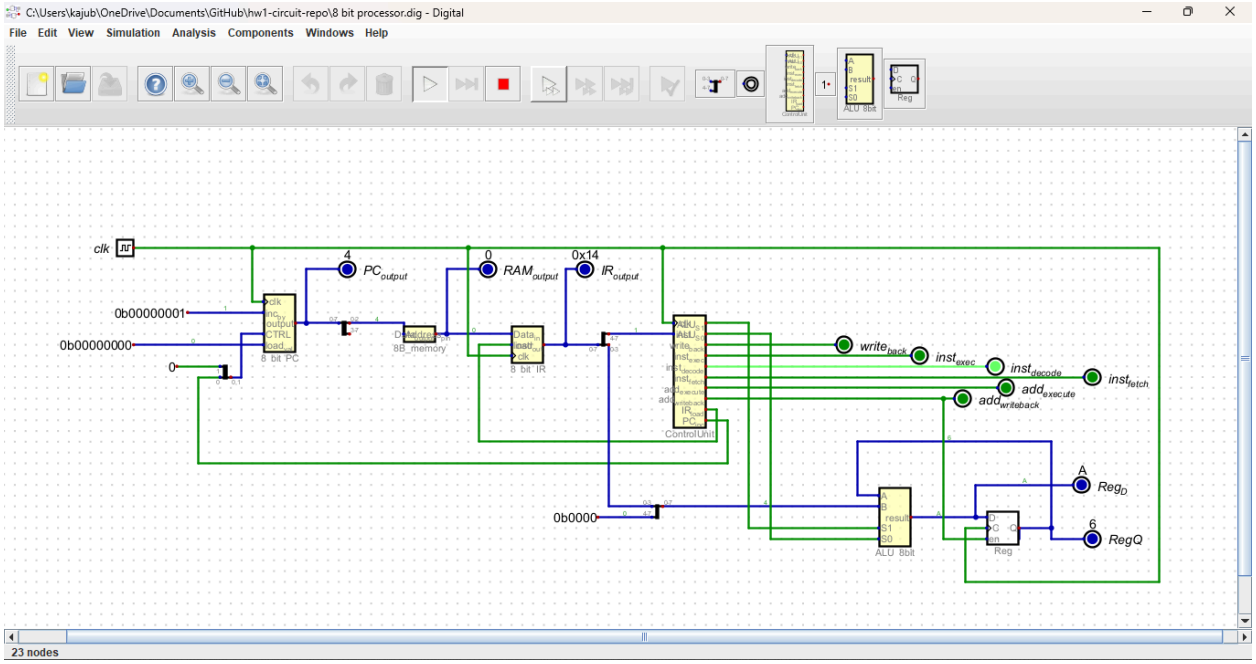
0 → 2 → 5 → 6 → 10

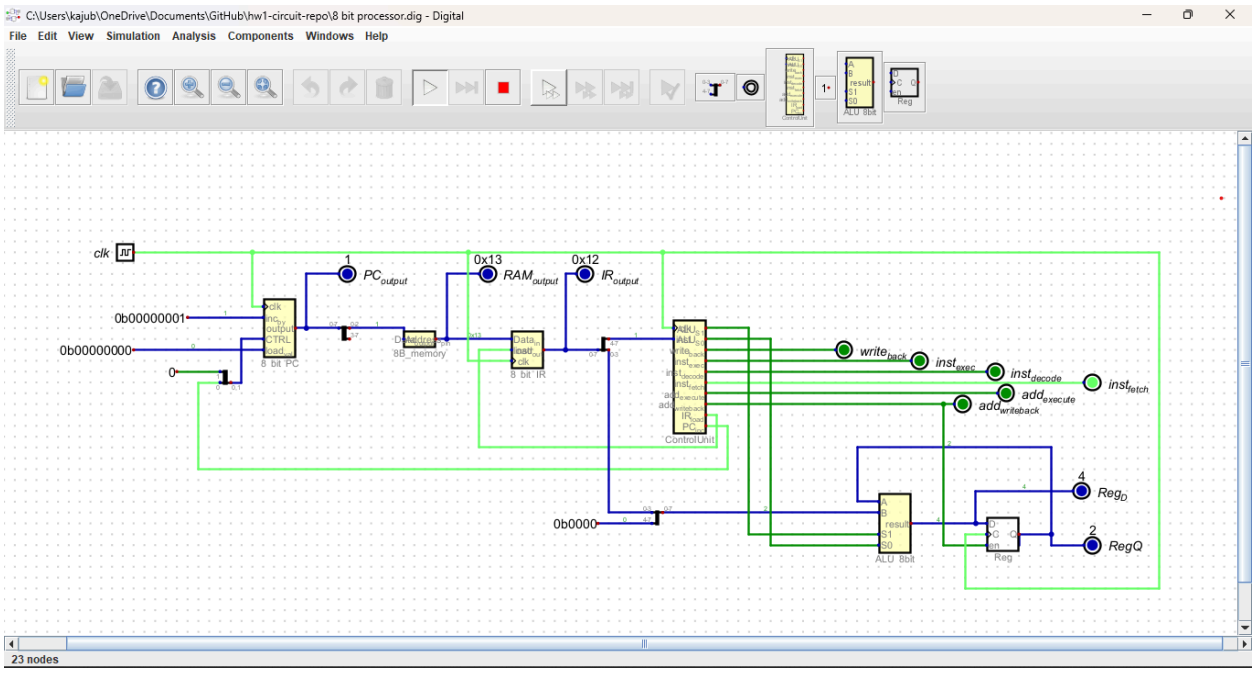
This matches the expected results:

- ADD 2 → 0 + 2 = 2
- ADD 3 → 2 + 3 = 5
- ADD 1 → 5 + 1 = 6
- ADD 4 → 6 + 4 = 10

These results confirm that:

- Instructions are fetched correctly
- The control unit properly sequences execution
- The ALU performs correct computations
- The register updates only during the writeback stage


C:\Users\kajub\OneDrive\Documents\GitHub\hw1-circuit-repo\8 bit processor.dig - Digital
File Edit View Simulation Analysis Components Windows Help
clk
0b00000001
0b00000000
0
4
PC_output
0
RAM_output
0x14
IR_output
8 bit PC
8B_memory
8 bit IR
ControlUnit
write_back
inst_exec
inst_decode
inst_fetch
add_execute
add_writeback
0b0000
ALU 8bit
Reg
A
Reg_D
6
RegQ
23 nodes


C:\Users\kajub\OneDrive\Documents\GitHub\hw1-circuit-repo\8 bit processor.dig - Digital
File Edit View Simulation Analysis Components Windows Help
clk
0b00000001
0b00000000
0
1
PC_output
0x13
RAM_output
0x12
IR_output
8 bit PC
8B_memory
8 bit IR
ControlUnit
write_back
inst_exec
inst_decode
inst_fetch
add_execute
add_writeback
0b0000
ALU 8bit
Reg
4
Reg_D
2
RegQ
23 nodes

8. Challenges and Debugging

Several challenges were encountered during development:

- Synchronizing the PC, IR, and control unit timing
- Preventing the PC from incrementing every clock cycle
- Ensuring instructions remained stable across multiple stages
- Correctly implementing writeback timing
- Debugging incorrect ALU and register connections

These issues were resolved by:

- Using ROM instead of RAM for stable instruction storage
- Introducing `PC_inc` to control when the PC increments
- Connecting `IR_load` to the fetch stage
- Carefully verifying signal timing and data flow

9. Conclusion

This project successfully demonstrates the design and implementation of an 8-bit multicycle processor. The processor correctly executes a sequence of instructions using a structured datapath and synchronized control unit.

The final implementation highlights key computer architecture concepts, including instruction execution timing, control signal generation, and datapath coordination.